

# Streaming Hit Predictor

Prediccion de Exito Comercial y Calidad Percibida en Catalogos de Streaming

## Proyecto Final

Introduccion a la Inteligencia Artificial

**Autor:** Sebastian Espinosa

**Fecha:** Mayo 2026

**Dataset:** 15.000 titulos, 10 plataformas (1980–2026)

### Resumen

Este proyecto aplica cuatro algoritmos de machine learning sobre un dataset de 15.000 titulos de plataformas de streaming para responder dos preguntas: que hace que un contenido sea un exito comercial, y si se puede predecir como lo va a calificar el publico. Los modelos supervisados (Arbol de Decision y SVM) predicen si un titulo supera la mediana de horas vistas, alcanzando mas del 91 % de accuracy. La Red Neuronal (implementada con TensorFlow) clasifica la calidad percibida en tres categorias, alcanzando F1-macro de 0.68 gracias a class weights balanceados y la inclusion de Rotten Tomatoes como feature adicional. El clustering con K-Means agrupa los titulos en cuatro perfiles.

## 1 Introduccion

---

La idea de este proyecto surgio de una pregunta concreta: se puede saber de antemano si una pelicula o serie va a tener exito? Las plataformas de streaming acumulan enormes cantidades de datos sobre sus catalogos, y parecia un caso de uso interesante para aplicar los modelos vistos en clase.

El objetivo principal fue construir modelos que predigan dos cosas distintas:

1. **Exito comercial** (`is_hit`): si un titulo supera la mediana de horas vistas en la plataforma.
2. **Calidad percibida** (`imdb_clase`): en que rango de rating de IMDb cae el titulo.

Ademas del objetivo predictivo, se aplico K-Means para segmentar el catalogo en perfiles sin usar etiquetas.

## 2 Dataset

---

### 2.1 Descripcion general

El dataset contiene informacion de 15.000 titulos distribuidos entre 10 plataformas de streaming: Netflix, Amazon Prime, Disney+, Hulu, HBO Max, Paramount+, Apple TV+, Peacock, JioCinema y Crunchyroll. Cubre el periodo 1980–2026 y originalmente tiene 25 columnas: metadata del titulo (tipo, genero, pais, idioma), metricas de recepcion (IMDb, Rotten Tomatoes, votos), informacion de produccion (presupuesto, premios) y la metrica objetivo: horas vistas en la plataforma.

Cuadro 1: Titulos por plataforma

| Plataforma   | Titulos       | Peliculas    | Series       | IMDb prom.  |
|--------------|---------------|--------------|--------------|-------------|
| Netflix      | 4.502         | 2.660        | 1.842        | 6.75        |
| Amazon Prime | 3.341         | 1.921        | 1.420        | 6.78        |
| Disney+      | 1.990         | 1.143        | 847          | 6.79        |
| Hulu         | 1.462         | 855          | 607          | 6.76        |
| HBO Max      | 1.228         | 700          | 528          | 7.04        |
| Paramount+   | 745           | 433          | 312          | 6.83        |
| Apple TV+    | 734           | 428          | 306          | 7.12        |
| Peacock      | 558           | 309          | 249          | 6.69        |
| JioCinema    | 296           | 178          | 118          | 6.77        |
| Crunchyroll  | 144           | 77           | 67           | 6.76        |
| <b>Total</b> | <b>15.000</b> | <b>8.704</b> | <b>6.296</b> | <b>6.79</b> |

## 2.2 Valores faltantes

Los valores nulos del dataset son estructurales, no errores de carga. Las columnas `num_seasons`, `num_episodes` y `duration_minutes` tienen 6.296 u 8.704 nulos, que coinciden con la cantidad de peliculas y series respectivamente. El presupuesto (`budget_million_usd`) tambien tiene nulos en la mayoria de las series. Imputar esos valores con la mediana global no tiene sentido; la solucion fue crear features derivados que incorporan esa logica.

## 3 Analisis Exploratorio

---

### 3.1 Variable objetivo: exito comercial

La mediana de horas vistas es **10 millones de horas**. Definir `is_hit` como superar la mediana produce exactamente 50 % de hits y 50 % de no-hits: no hay desbalanceo. La tasa de exito es consistente entre plataformas (todas rondan el 50 %).

### 3.2 Variable objetivo: calidad percibida

Para `imdb_clase` use los percentiles 25 y 75 del rating de IMDb como cortes. La distribucion queda:

- **Bajo:** 28 % de los titulos
- **Medio:** 50 % de los titulos
- **Alto:** 22 % de los titulos

### 3.3 Correlaciones relevantes

La correlación más alta con las horas vistas la tiene `votos_log` (transformación logarítmica de los votos de IMDb), con un valor de **0.45**. El resto de las variables numéricas tiene correlaciones menores a 0.15. Esto anticipaba que los modelos de clasificación de éxito comercial iban a depender fuertemente de esa variable.

### 3.4 Distribución por género y plataforma

Los géneros con mayor promedio de horas vistas son Acción y Ciencia Ficción. Los documentales tienen mejor rating de IMDb en promedio pero menos horas vistas. HBO Max y Apple TV+ tienen el IMDb promedio más alto, aunque también tienen el catálogo más pequeño.

## 4 Preprocesamiento y Feature Engineering

---

### 4.1 Features contruidos

Empezando con las 25 columnas originales, construí 11 features adicionales:

| Cuadro 2: Features contruidos   |                                                                                                         |
|---------------------------------|---------------------------------------------------------------------------------------------------------|
| Feature                         | Cálculo                                                                                                 |
| <code>es_pelicula</code>        | 1 si <code>type == "Movie"</code> , 0 si serie                                                          |
| <code>cantidad_generos</code>   | Cantidad de géneros listados en la columna <code>genres</code>                                          |
| <code>tamano_elenco</code>      | Cantidad de actores en el elenco                                                                        |
| <code>antiguedad</code>         | $2026 - \text{release\_year}$                                                                           |
| <code>es_reciente</code>        | 1 si $\text{release\_year} \geq 2020$                                                                   |
| <code>mes_agregado</code>       | Mes en que fue agregado a la plataforma                                                                 |
| <code>dias_en_plataforma</code> | Días transcurridos desde que fue agregado                                                               |
| <code>votos_log</code>          | $\ln(1 + \text{imdb\_votes})$                                                                           |
| <code>presupuesto_log</code>    | $\ln(1 + \text{budget})$ , 0 si nulo                                                                    |
| <code>tiene_presupuesto</code>  | 1 si el presupuesto está reportado, 0 si no                                                             |
| <code>duracion_total</code>     | Películas: <code>duration_minutes</code> ; Series: <code>num_episodes</code><br>$\times 45 \text{ min}$ |

La transformación logarítmica de votos y presupuesto comprime valores extremos, muy frecuentes en entretenimiento: una película de Marvel puede tener cientos de millones de votos mientras que un documental niche puede tener cientos.

### 4.2 Codificación de categóricas

Las variables categóricas de baja cardinalidad (`platform`, `primary_genre`, `rating`) se codificaron con `LabelEncoder`.

### 4.3 Sets de features y splits

Se prepararon dos sets de features distintos:

- **FEATURES\_CLF** (17 variables): Para Arbol de Decision y SVM. Incluye `rotten_tomatoes_score` como predictor de exito comercial.
- **FEATURES\_NN** (17 variables): Para la Red Neuronal. Incluye `rotten_tomatoes_score` como predictor (calificacion de criticos, distinta al rating de audiencia IMDb que es el target).

El split fue 80/20 estratificado con `random_state=42`: 12.000 filas de entrenamiento y 3.000 de prueba. Ambos sets se normalizaron con `StandardScaler` ajustado solo sobre el entrenamiento, para no filtrar informacion del test al entrenamiento.

## 5 Modelos de Machine Learning

---

### 5.1 Arbol de Decision

#### 5.1.1. Configuracion

Para buscar los mejores hiperparametros use `GridSearchCV` con 5-fold cross-validation, optimizando F1-score:

- `max_depth`: [3, 5, 7, 10, None]
- `min_samples_split`: [2, 10, 30]
- `criterion`: [gini, entropy]

Mejores hiperparametros: `max_depth=None`, `min_samples_split=2`, `criterion=gini`. Sin embargo, el arbol resultante solo tiene profundidad real de 3 y 8 hojas, lo que indica que los datos son separables con pocas decisiones.

#### 5.1.2. Resultados

Cuadro 3: Metricas del Arbol de Decision (test set)

| Metrica   | Valor |
|-----------|-------|
| Accuracy  | 0.914 |
| Precision | 0.915 |
| Recall    | 0.914 |
| F1-Score  | 0.914 |
| ROC-AUC   | 0.972 |

### 5.1.3. Hallazgo clave: dominancia de `votos_log`

Al revisar las importancias de features, `votos_log` tiene importancia = 1.0. Las otras 16 variables contribuyen practicamente nada. Esto se explica por la correlacion estructural del dataset:

- `imdb_votes` (raw) y `hours_watched_million` tienen correlacion **0.90** en este dataset.
- Tras la transformacion logaritmica, `votos_log` tiene correlacion 0.45 con las horas vistas pero correlacion **0.77 con `is_hit`** (target binario).
- Los arboles de decision usan umbrales. Con 3 niveles el arbol identifica cortes en `votos_log` que separan hits de no-hits con 91 % de precision.

**No es data leakage:** `imdb_votes` viene de IMDb y `hours_watched_million` es la metrica interna de la plataforma. Son columnas independientes con alta correlacion estructural en este dataset sintetico. La conclusion es que la cantidad de votos que un titulo genera en IMDb un acercamiento robusto de su popularidad comercial.

## 5.2 Support Vector Machine (SVM)

### 5.2.1. Configuracion

El SVM con kernel RBF es computacionalmente costoso: en 12.000 filas el grid search tardaria horas. La solucion fue hacer el grid search sobre una muestra aleatoria de 5.000 filas del entrenamiento, y luego re-entrenar el modelo con los mejores parametros sobre el dataset completo.

Grid buscado:

- `kernel`: [linear, rbf]
- `C`: [0.1, 1, 10]
- `gamma`: [scale, 0.01, 0.1]

Mejores hiperparametros: `kernel=rbf`, `C=1`, `gamma=0.01`.

### 5.2.2. Resultados

Cuadro 4: Metricas del SVM (test set)

| Metrica   | Valor |
|-----------|-------|
| Accuracy  | 0.912 |
| Precision | 0.912 |
| Recall    | 0.912 |
| F1-Score  | 0.912 |
| ROC-AUC   | 0.975 |

El SVM tiene resultados muy similares al Arbol de Decision. Su ROC-AUC es superior (0.975 vs 0.972). Como el SVM no tiene importancias de features directas, se calculo permutation importance: `votos_log` tambien domina, confirmando el hallazgo del arbol.

### 5.3 Red Neuronal (MLP)

#### 5.3.1. Configuracion

La red neuronal resuelve un problema diferente: clasificacion multiclase del rating de IMDb en tres categorias. Se implemento con TensorFlow/Keras incluyendo `rotten_tomatoes_score` (puntuacion de criticos) como feature; `imdb_rating` sigue excluido para evitar leakage directo.

Arquitectura: `Dense(128) ->Dropout(0.3) ->Dense(64) ->Dropout(0.2) ->Dense(3, softmax)`. Se usaron class weights balanceados para corregir el sesgo hacia la clase Medio (50 % del dataset), con `EarlyStopping(patience=20)` y `ReduceLROnPlateau`. El modelo convergio en 43 epocas.

#### 5.3.2. Resultados

Cuadro 5: Metricas del MLP por clase (test set)

| Clase     | Precision | Recall | F1           | Soporte |
|-----------|-----------|--------|--------------|---------|
| Bajo      | 0.63      | 0.83   | 0.71         | 830     |
| Medio     | 0.74      | 0.55   | 0.63         | 1.504   |
| Alto      | 0.66      | 0.78   | 0.71         | 666     |
| Macro avg | 0.67      | 0.72   | <b>0.68</b>  | 3.000   |
| Accuracy  |           |        | <b>0.676</b> |         |

#### 5.3.3. Impacto de class weights y Rotten Tomatoes

El MLPClassifier anterior obtenia F1-macro de 0.36 porque predecia casi exclusivamente la clase Medio (recall Bajo = 0.01). Con TensorFlow y class weights balanceados, las tres clases reciben atencion equitativa: Bajo pasa de recall 0.01 a 0.83, Alto de 0.28 a 0.78. Incluir `rotten_tomatoes_score` como feature (correlacionado con IMDb pero medido por criticos independientes) apporto la senal mas util para separar las categorias de calidad.

### 5.4 K-Means (Clustering)

#### 5.4.1. Seleccion del numero de clusters

Se probaron valores de  $k$  de 2 a 10 usando tres criterios simultaneamente:

Cuadro 6: Criterios de seleccion de  $k$  (extracto)

| $k$      | Inercia       | Silhouette   | Davies-Bouldin |
|----------|---------------|--------------|----------------|
| 2        | 100.983       | 0.170        | 2.034          |
| 3        | 88.678        | 0.174        | 1.736          |
| <b>4</b> | <b>78.793</b> | <b>0.181</b> | <b>1.449</b>   |
| 5        | 71.255        | 0.174        | 1.381          |
| 6        | 64.483        | 0.184        | 1.303          |

Se eligio  $k = 4$  porque es el primer maximo local del silhouette score. A partir de  $k = 5$  el silhouette cae y los clusters se vuelven dificiles de diferenciar semanticamente.

#### 5.4.2. Perfiles de clusters

Cuadro 7: Valores medios por cluster (features representativas)

| Feature           | Blockbuster | Nicho | Joya Critica | Promedio |
|-------------------|-------------|-------|--------------|----------|
| Titulos           | 134         | 4.826 | 4.104        | 5.936    |
| IMDb              | 6.74        | 6.41  | 8.05         | 6.28     |
| RT Score          | 66.09       | 62.99 | 82.27        | 61.19    |
| Horas vistas (M)  | 2.455       | 48.3  | 52.0         | 50.3     |
| Presupuesto (log) | 1.75        | 0.00  | 1.70         | 2.87     |
| Premios ganados   | 0.34        | 0.07  | 0.98         | 0.04     |

Los cuatro perfiles son bastante claros:

- **Blockbuster Costoso** (134 titulos): Horas vistas promedio de 2.455 millones, 50 veces mas que el resto. IMDb normal (6.74): el exito masivo no requiere calidad excepcional.
- **Nicho Subestimado** (4.826 titulos): Sin presupuesto reportado, pocas horas vistas, IMDb bajo. Producciones pequenas sin traccion masiva.
- **Joya de Critica** (4.104 titulos): IMDb de 8.05 y RT Score de 82, los mas altos de todos los clusters. Buena cantidad de premios. Horas vistas modestas: calidad percibida y exito comercial son cosas distintas.
- **Contenido Promedio** (5.936 titulos): Cerca de la media en todo. La clase mayoritaria del dataset.

## 6 Comparacion de Modelos



Cuadro 8: Comparacion de modelos supervisados

| Modelo            | Target     | Accuracy | F1     | ROC-AUC |
|-------------------|------------|----------|--------|---------|
| Arbol de Decision | is_hit     | 0.914    | 0.914  | 0.972   |
| SVM               | is_hit     | 0.912    | 0.912  | 0.975   |
| MLP (TensorFlow)  | imdb_clase | 0.676    | 0.684* | —       |

\* F1-macro. El F1-weighted es 0.43.

Para `is_hit`: el SVM tiene ROC-AUC superior (0.975 vs 0.972), pero la diferencia no es significativa. El Arbol de Decision tiene la ventaja de ser completamente interpretable: se puede trazar exactamente que valor de `votos_log` separa un hit de un no-hit. Para produccion elegiria el Arbol de Decision por su interpretabilidad, dado que la diferencia de rendimiento frente al SVM es casi nula.

## 7 Conclusiones

---

El proyecto muestra resultados interesantes en dos direcciones distintas.

Por un lado, **predecir exito comercial es factible con buena precision** (91 %). El factor dominante es la cantidad de votos que un titulo acumula en IMDb, que funciona como factor de popularidad. Tiene sentido: los titulos que generan discusion publica son los que la gente busca calificar, y tambien son los que mas se consumen en las plataformas.

Predecir **calidad percibida** es mas desafiante pero manejable con las tecnicas correctas. Con TensorFlow, class weights balanceados y la inclusion de Rotten Tomatoes como feature, la red neuronal alcanza F1-macro de 0.68 y accuracy de 0.68, mejorando un 88 % respecto al MLPClassifier inicial (F1-macro 0.36).

El clustering con K-Means revela algo que no es obvio a priori: los titulos de mayor calidad percibida (Joya de Critica) y los de mayor exito comercial (Blockbuster Costoso) son grupos practicamente separados. Los titulos que todos ven no son necesariamente los que mejor se califican, y viceversa.

### 7.1 Posibles mejoras

- Agregar features de texto: resumen del titulo, generos mas especificos, codificacion semantica del director y elenco.
- Probar modelos de boosting (XGBoost, LightGBM) para comparar contra SVM.
- Para el clustering, experimentar con DBSCAN que no requiere especificar  $k$  a priori.